



Cyberscope

A *TAC Security* Company

Audit Report **wCMXR Token**

May 2026

Network BSC

Address 0x8366e68e9d33492ec8c646d3674f106168e4969b

Audited by © cyberscope

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Multisign
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Multisign
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Multisign

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	CCR	Contract Centralization Risk	Multisign
●	MC	Missing Check	Unresolved
●	RPF	Redundant Pausable Functionality	Unresolved
●	BFR	Burn Function Restriction	Unresolved
●	MISC	Missing Initial Supply Check	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved

Table of Contents

Analysis	1
Diagnostics	2
Table of Contents	3
Risk Classification	5
Review	6
Audit Updates	6
Source Files	6
Findings Breakdown	7
ST - Stops Transactions	8
Description	8
Recommendation	9
MT - Mints Tokens	10
Description	10
Recommendation	10
BC - Blacklists Addresses	11
Description	11
Recommendation	12
BFR - Burn Function Restriction	13
Description	13
Recommendation	13
CCR - Contract Centralization Risk	14
Description	14
Recommendation	14
MC - Missing Check	15
Description	15
Recommendation	15
MISC - Missing Initial Supply Check	16
Description	16
Recommendation	16
RPF - Redundant Pausable Functionality	17
Description	17
Recommendation	17
L04 - Conformance to Solidity Naming Conventions	18
Description	18
Recommendation	19
Functions Analysis	20
Inheritance Graph	21
Flow Graph	22
Summary	23

Disclaimer**24****About Cyberscope****25**

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	wrapped CMXR Token
Explorer	https://bscscan.com/address/0x8366e68e9d33492ec8c646d3674f106168e4969b
Symbol	wCMX-R
Decimals	18
Total Supply	1,000,000,000

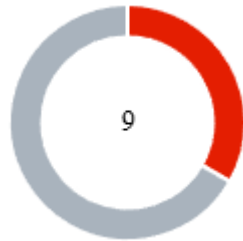
Audit Updates

Initial Audit	14 May 2026
----------------------	-------------

Source Files

Filename	SHA256
wCMXR.sol	339920dbb2147c1fc1ef229f4eb7f8bef15360ff98d4c7cb2a30c387dbb5f5bb

Findings Breakdown



● Critical	3
● Medium	0
● Minor / Informative	6

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	3
● Medium	0	0	0	0
● Minor / Informative	5	0	0	1

ST - Stops Transactions

Criticality	Critical
Location	wCMXR.sol#L633,651
Status	Multisign

Description

The contract owner has the authority to stop the transactions for all users. As described in the `BC` finding, the owner may take advantage of it by calling the `updateBlacklist` function on the liquidity pair address. When the liquidity pair address is blacklisted, every buy transaction reverts on the `from` check and every sell transaction reverts on the `to` check, halting all trading against the pool.

```
function updateBlacklist(address account, bool status) public onlyOwner
{
    _isBlacklisted[account] = status;
    emit BlacklistUpdated(account, status);
}

function _beforeTokenTransfer(address from, address to, uint256 amount)
internal virtual override (ERC20, ERC20Pausable){
    require(!_isBlacklisted[from], "wCMXR: sender is blacklisted");
    require(!_isBlacklisted[to], "wCMXR: recipient is blacklisted");
    super._beforeTokenTransfer(from, to, amount);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

MT - Mints Tokens

Criticality	Critical
Location	wCMXR.sol#L645
Status	Multisign

Description

The contract owner has the authority to mint tokens. The owner may take advantage of it by calling the `mint` function. As a result, the contract tokens will be highly inflated.

```
function mint(uint256 amount) public onlyOwner returns (bool)
{
    _mint(msg.sender, amount);
    return true;
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

BC - Blacklists Addresses

Criticality	Critical
Location	wCMXR.sol#L633,651
Status	Multisign

Description

The contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `updateBlacklist` function. The `updateBlacklist` function allows the contract owner to set the blacklisted status of any address. The `_beforeTokenTransfer` hook enforces `require(!_isBlacklisted[from])` and `require(!_isBlacklisted[to])`, so a blacklisted address is blocked from both sending and receiving tokens. The function applies no restrictions on which addresses can be blacklisted

```
function updateBlacklist(address account, bool status) public onlyOwner {
    _isBlacklisted[account] = status;
    emit BlacklistUpdated(account, status);
}

function _beforeTokenTransfer(address from, address to, uint256 amount)
internal virtual override (ERC20, ERC20Pausable){
    require(!_isBlacklisted[from], "wCMXR: sender is blacklisted");
    require(!_isBlacklisted[to], "wCMXR: recipient is blacklisted");
    super._beforeTokenTransfer(from, to, amount);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

BFR - Burn Function Restriction

Criticality	Minor / Informative
Location	wCMXR.sol#L651
Status	Unresolved

Description

The `_beforeTokenTransfer` hook enforces the blacklist check on the `from` address for every transfer, mint, and burn operation. The inherited `burn` function from `ERC20Burnable` calls `_burn(msg.sender, amount)`, which routes through `_beforeTokenTransfer(msg.sender, address(0), amount)`. If the caller is blacklisted, the require statement `require(!_isBlacklisted[from], ...)` will revert, preventing the user from destroying their own tokens. The blacklist mechanism is intended to restrict transfers, but it also unintentionally suspends blacklisted users from burning their own balance.

```
function _beforeTokenTransfer(address from, address to, uint256 amount)
internal virtual override (ERC20, ERC20Pausable){
    require(!_isBlacklisted[from], "wCMXR: sender is blacklisted");
    require(!_isBlacklisted[to], "wCMXR: recipient is blacklisted");
    super._beforeTokenTransfer(from, to, amount);
}
```

Recommendation

The team is advised to allow burn operations to bypass the blacklist check, by exempting transfers where the `to` address is the zero address from the `_isBlacklisted[from]` check.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	wCMXR.sol#L633,645
Status	Multisign

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function updateBlacklist(address account, bool status) public onlyOwner
{...}

function mint(uint256 amount) public onlyOwner returns (bool) {...}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

MC - Missing Check

Criticality	Minor / Informative
Location	wCMXR.sol#L633
Status	Unresolved

Description

The `updateBlacklist` function does not validate the `account` argument against the zero address. If the owner sets the blacklisted status of `address(0)` to true, the `_beforeTokenTransfer` hook will revert on every subsequent mint and burn operation.

```
function updateBlacklist(address account, bool status) public onlyOwner
{
    _isBlacklisted[account] = status;
    emit BlacklistUpdated(account, status);
}
```

Recommendation

The team is advised to add a require statement in the `updateBlacklist` function to ensure that the `account` argument is not the zero address.

MISC - Missing Initial Supply Check

Criticality	Minor / Informative
Location	wCMXR.sol#L625
Status	Unresolved

Description

The contract is processing the `initialSupply` argument that has not been properly sanitized and checked that it forms the proper shape. The constructor accepts the `initialSupply` as a raw `uint256` value and passes it directly to `_mint` without any validation that it is properly scaled according to the token's 18 decimals. As a result, the variable may produce vulnerability.

```
constructor(uint256 initialSupply) ERC20("wrapped CMXR Token", "wCMX-R")
{
    _mint(msg.sender, initialSupply);
}
```

Recommendation

The team is advised to properly check the `initialSupply` argument according to the required specifications, ensuring it is scaled by the token's decimals and bounded by a reasonable maximum value.

RPF - Redundant Pausable Functionality

Criticality	Minor / Informative
Location	wCMXR.sol#L625
Status	Unresolved

Description

The contract inherits from the `ERC20Pausable` and `Pausable` abstract contracts to support an emergency-stop mechanism. In smart contracts, the pausable functionality typically allows an authorized account to halt token transfers through external `pause` and `unpause` functions. However, in this case, while the contract inherits the pausable functionality, it does not expose any external functions to invoke the internal `_pause` and `_unpause` methods. Therefore, the inheritance of `ERC20Pausable` and `Pausable` is redundant.

```
contract wCMXR is ERC20, ERC20Pausable, Ownable, ERC20Burnable {

    function _pause() internal virtual whenNotPaused {
        _paused = true;
        emit Paused(_msgSender());
    }

    function _unpause() internal virtual whenPaused {
        _paused = false;
        emit Unpaused(_msgSender());
    }
}
```

Recommendation

The team is advised to remove the `ERC20Pausable` and `Pausable` inheritance, since eliminating redundancies will reduce code size and enhance readability.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	wCMXR.sol#L618
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
contract wCMXR is ERC20, ERC20Pausable, Ownable, ERC20Burnable {
    // Mapping to store blacklisted status
    mapping(address => bool) private _isBlacklisted;

    // Event emitted when an account is blacklisted or removed
    event BlacklistUpdated(address indexed account, bool isBlacklisted);
    ...
    // Logic to prevent blacklisted addresses from sending or
    receiving
    require(!_isBlacklisted[from], "wCMXR: sender is blacklisted");
    require(!_isBlacklisted[to], "wCMXR: recipient is blacklisted");

    super._beforeTokenTransfer(from, to, amount);
}
}
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

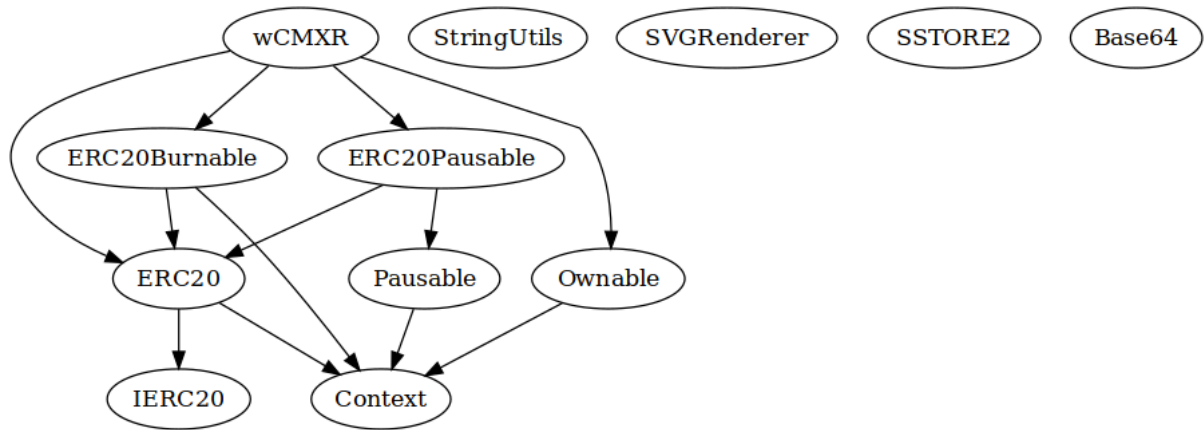
Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

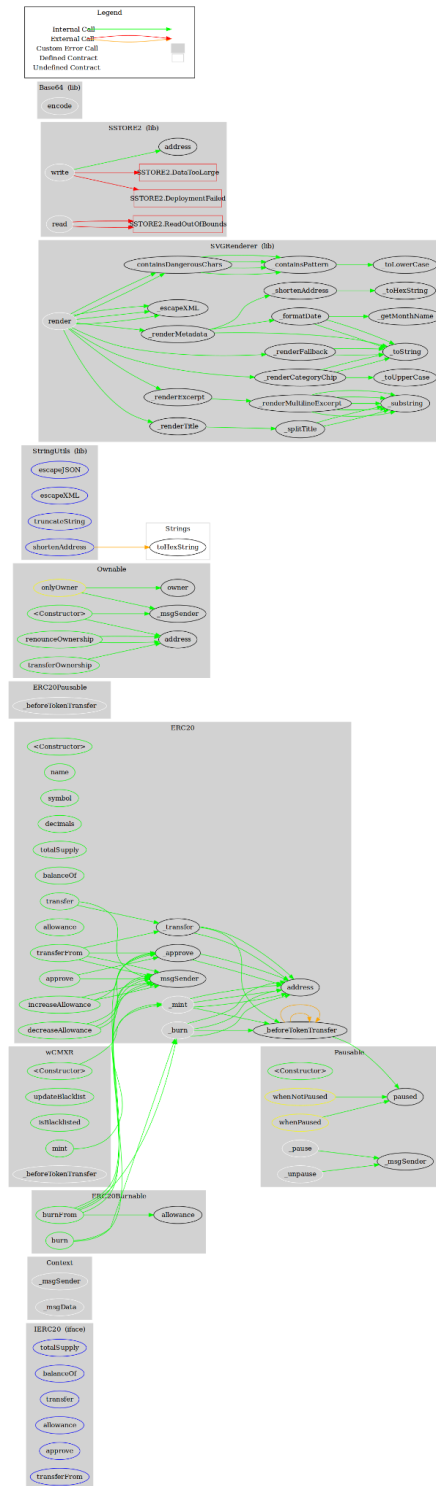
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
wCMXR	Implementation	ERC20, ERC20Pausable, Ownable, ERC20Burnable		
		Public	✓	ERC20
	updateBlacklist	Public	✓	onlyOwner
	isBlacklisted	Public		-
	mint	Public	✓	onlyOwner
	_beforeTokenTransfer	Internal	✓	

Inheritance Graph



Flow Graph



Summary

CRYMADX Ecosystem / CRYMAD Chain contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like mint tokens and massively blacklist addresses. A multi-wallet signing pattern is used but this does not eliminate the thread. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io